

AI 활용 기술 문서 — INCANT

제출물 ④ · NHN 2026 게임 AI 해커톤 INCANT: "말이 곧 마법" — 플레이어가 자유롭게 타이핑한 문장을 LLM이 실시간으로 해석해 주문으로 바꾸는 로그라이크.

이 문서는 대회 제출 요건 ④(AI 활용 기술적 설명 / AI 도구·활용 내역 / 외부 에셋·오픈소스 출처)에 대응한다.

목차

- **1부. 게임 속 AI** — 제품에 탑재된 AI의 구조와 프롬프트
 - 1.1 개요: LLM이 게임에서 맡는 일
 - 1.2 판정 파이프라인
 - 1.3 판정 프롬프트 핵심 규칙
 - 1.4 "LLM을 신뢰하지 않는" 설계
 - 1.5 표현 DSL: 소환 행동·벽 형상
 - 1.6 복합 영창 시퀀스
 - 1.7 기억하는 보스: 도발 대사 생성
 - 1.8 진화·융합 작명
 - 1.9 인프라: Cloudflare Worker 프록시
 - **2부. AI로 만든 게임** — 개발에 사용한 AI 도구와 활용 내역
 - 2.1 사용한 AI 도구
 - 2.2 협업 워크플로
 - 2.3 대표 사례: 프롬프트에서 산출물까지
 - 2.4 배운 점: 에이전트 지휘 노하우
 - **3부. 외부 에셋·오픈소스 출처**
 - 3.1 오디오 / 3.2 이미지 / 3.3 폰트 / 3.4 자체 제작 / 3.5 오픈소스
-

1부. 게임 속 AI

1.1 개요 — LLM이 게임에서 맡는 일

INCANT의 핵심 경험은 **정해진 스킬 목록이 없다는 것**이다. 플레이어는 "얼음 감옥으로 가둬라", "숲의 분노", "lightning storm", "배고프다" 같은 **어떤 문장이든** 입력할 수 있고, 게임은 그 의미를 해석해 원소·형태·위력·효과를 갖춘 실제 주문으로 변환한다.

이 변환의 심장이 ****주문 판정기(judge)****다. 판정기는 자유 텍스트를 받아 게임 엔진이 소비할 수 있는 **구조화된 JSON**을 돌려준다.

```
플레이어 입력: "얼음 가시로 궤뿔어라"
  |
  ▼ (LLM 판정)
구조화 판정: {
  disposition: "cast",
  spell: {
    name: "빙결의 가시", effect: "damage", target: "enemy",
    element_primary: "ice", form: "bolt", size: "medium",
    status: ["freeze"], power: 62, cost: 40
  }
}
  |
  ▼
게임 엔진: 얼음 속성 · bolt 형태 · 62 위력 · 빙결 상태이상 투사체를 생성
```

판정에 사용하는 모델은 **Google Gemini 3.5 Flash-Lite**이며, 클라우드플레어 워커 프록시를 통해 호출한다. 판정 외에도 두 곳에서 LLM을 쓴다 — **보스의 도발 대사 생성(1.7)**과 **주문 진화·정령 융합 시의 작명(1.8)**.

설계 원칙: LLM은 "의미를 이해하는 번역기"로만 쓰고, **게임 밸런스·안전·재현성은 코드가 통제한다**. LLM의 출력은 항상 검증·클램프를 거치며, 실패해도 게임은 멈추지 않는다(1.4).

1.2 판정 파이프라인

판정 한 번은 다음 체인을 거친다. 각 단계는 "LLM에 도달하기 전에 최대한 거르고, LLM이 실패해도 게임은 계속 돌게" 설계됐다. (구현: [src/spell/geminiJudge.ts](#))

```
judge(text)
|
├─ ① 로컬 사전검사 (precheck) — 명백한 년센스·차단어는 LLM 호출 없이 즉시 처리
|   └─ mockJudge.precheckText() source: 'local'
|
├─ ② localStorage 캐시 조회 — 같은 문장 = 같은 판정 (재현성·속도·호출량 절감)
|   └─ 키: incant:judge:v2:meaning-v2.6-seq:<문장> source: 'cache'
|
├─ ③ 프록시 요청 (2.5초 타임아웃) — Cloudflare Worker → Gemini
|   └─ fetch({ text }), AbortController 2500ms source: 'gemini'
|
├─ ④ 스키마 재검증 (validateJudgement) — 스키마 밖 값은 거부·클램프
|   └─ 유효하면 캐시에 저장 (장애 폴백은 저장하지 않음)
|
└─ ⑤ 폴백 (MockJudge) — 타임아웃·네트워크 오류·무효 응답 시
    └─ 게임은 절대 멈추지 않는다 source: 'fallback'
```

파이프라인 설계점

- **타임아웃 2.5초** (`TIMEOUT_MS = 2500`): 판정이 늦으면 전투 리듬이 깨지므로, 2.5초를 넘기면 즉시 로컬 폴백으로 전환한다.
- **캐시는 유효 결과만 저장**: 장애로 인한 폴백 판정은 캐시하지 않는다. 프록시가 복구되면 다음 시전에서 진짜 판정을 받는다.
- **출처 추적** (`lastSource`): 각 판정이 `local / cache / gemini / fallback` 중 어디서 왔는지 기록해, 개발 중 폴백 빈도를 관찰할 수 있게 했다.

1.3 판정 프롬프트 핵심 규칙

프롬프트는 **서버(프록시)에 고정**한다. 클라이언트는 `{ text }` (플레이어 입력)만 보내고, 판정 규칙·출력 스키마는 워커가 강제한다 — 프롬프트를 클라이언트에 두면 조작이 가능하기 때문이다(1.9).

프롬프트는 LLM에게 **정해진 순서로 판단**하도록 지시한다. 다음은 실제 배포 프롬프트의 핵심 규칙 발췌다 (`proxy/worker.js`, 버전 `meaning-v2.6-seq`):

당신은 자유 텍스트 마법 게임의 의미 판정관이다. 반드시 JSON 하나만 출력한다.

다음 순서를 지켜 판단한다.

1. 입력 언어와 표현의 실제 의미를 파악한다. 외래어와 비유도 번역해 이해한다.
2. `disposition`을 결정한다.
 - `cast`: 의미가 있는 모든 문장. 마법 단어가 없어도 창의적인 약한 효용 주문으로 번역한다.
 - `fizzle`: "ㅇㄴㅇㄹ", "asdf"처럼 의미 없는 키보드 매시만 해당한다.
 - `blocked`: 욕설, 혐오, 노골적 유해 표현 등 부적절한 입력만 해당한다.
3. `cast`라면 `effect`와 `target`을 먼저 결정한 뒤 `element`와 `form`을 시각적 은유로 고른다.
 - `effect`: `damage|heal|shield|buff|control|summon`
 - `target`: `enemy|self|area`
 - "배고프다", "피곤하다"처럼 상태를 말하는 문장은 `heal` 또는 `buff/self`로 해석한다.
 - "나를 지켜줘"는 `shield/self`, "숲의 분노"는 `damage` 또는 `control/area`로 해석한다.
 - "라이트닝 스톰"과 "lightning storm"은 번개 폭풍의 동일한 의미로 해석한다.
4. `power`와 `cost`를 정한다.
 - 구체적·창의적·서사적인 묘사: `power 60~100`
 - 단순한 마법 표현: `power 30~50`
 - 마법과 무관하지만 의미 있는 문장: `power 15~40`
 - 표기 언어(한국어·외래어·영어)는 `power`에 영향을 주지 않는다. 번역했을 때 의미와 구체성이 같으면 `power`도 동일해야 한다.
 - 창의성 가정은 표현의 구체성·서사성에만 근거한다. 단순 마법 단어에는 가정하지 않는다.
 - `cost`는 `power`의 0.5~0.7배이며 1~100이다.
5. `effect`가 `summon`인 주문에 한해, 소환수 움직임 묘사를 `behavior`(움직임 프로그램)로 설계한다. (→ 1.5에서 상술)
6. `form`이 `wall`인 주문에 한해, 벽의 모양 묘사가 있으면 `shape`로 설계한다. 모양 묘사가 없으면 `shape`를 넣지 않고 기본 원호로 폴백한다. (→ 1.5)
7. 시간 순서가 있는 복합 동작이나 동시 다원소 동작이면 `spell_plan`만 설계한다. 단일 동작이면 `spell`만 내고 `spell_plan`은 넣지 않는다. (→ 1.6)

이 프롬프트가 담은 설계 의도:

- **의미 우선, 단어 매칭 아님** (1단계): "`화염구`" 든 "`fireball`" 이든 "`불덩이를 던져라`" 든 같은 의미로 해석한다. 외래어·비유·다국어 모두 번역해 이해하도록 지시했다.
- **넌센스와 유해 입력만 거부** (2단계): 의미가 조금이라도 있으면 `cast` 한다. 키보드 매시("`asdf`")만 `fizzle`, 욕설·혐오만 `blocked`. "우리가 미리 정한 마법 단어"에 갇히지 않게 하는 것이 핵심.
- **언어 중립 위력** (4단계): 한국어로 쓰든 영어로 쓰든, **의미와 구체성이 같으면 power도 같아야 한다**. 특정 언어 사용자가 유·불리하지 않도록 명시했다. (이 규칙은 held-out 검증으로 교차언어 격차를 5.0→3.3으로 줄인 튜닝의 결과다 — 2부에서 상술)
- **창의성은 서사성에 가점** (4단계): 단순히 마법 단어를 넣었다고 위력이 오르지 않는다. 구체적·서사적 묘사에만 가점해, 플레이어가 "문장을 잘 짓는" 방향으로 유도한다.
- **복합성은 시간축으로 번역** (7단계): 긴 문장 자체가 아니라 실제 순차·동시 의미가 있을 때만 `spell_plan`을 만든다. LLM은 안무를 고르고, 로컬은 시간·Power·Mana 예산을 다시 계산한다.

1.4 "LLM을 신뢰하지 않는" 설계

LLM의 출력은 **신뢰할 수 없는 입력**으로 취급한다. 게임의 밸런스·안전·재현성이 모델의 번덕에 좌우되면 안 되기 때문이다. 세 겹의 방어가 있다.

① **스키마 재검증·클램프** (`src/spell/validate.ts`) 프록시가 돌려준 JSON을 게임이 그대로 쓰지 않는다.

`validateJudgement`가 다시 한번 검증해 — `effect/element/form`이 허용 목록(enum) 안에 있는지, `power/cost`가 범위 안인지, 소환 behavior의 수치가 상한을 넘지 않는지 — 확인하고, 벗어난 값은 거부하거나 클램프한다. LLM이 `power: 9999`를 뱉어도 게임에는 반영되지 않는다.

② **서버측 프롬프트 고정** (`proxy/worker.js`) 프롬프트와 API 키는 워커에 있고, 클라이언트는 `{ text }`만 보낸다. 플레이어가 브라우저에서 프롬프트를 바꿔 "power 100 고정" 같은 조작을 하는 것을 원천 차단한다.

③ **무중단 폴백** (`src/spell/mockJudge.ts`) 프록시 장애·타임아웃·무료 티어 한도 소진(429) 등 어떤 이유로든 원격 판정이 실패하면, 로컬 규칙 기반 판정기 `MockJudge`가 즉시 대신한다. 판정 품질은 다소 낮아질 수 있어도 **게임은 절대 멈추지 않는다**. 심사위원이 몰려 무료 한도를 소진해도 게임은 계속 플레이 가능하다.

요약하면 LLM은 **의미 해석**만 담당하고, 그 출력의 안전·범위·재현성은 전부 결정론적 코드가 통제한다. "창의성은 LLM에, 밸런스는 코드에."

1.5 표현 DSL — 소환 행동·벽 형상

가장 앞선 활용은 **LLM이 소환수의 움직임을 직접 설계**하는 것이다. "**분신을 만들어 지그재그로 돌진시켜라**" 같은 입력에서, LLM은 소환 주문임을 인식하고 ****움직임 프로그램(behavior)****을 함께 생성한다.

움직임은 6개의 부품(kind)으로 이루어진 작은 DSL로 표현된다:

kind	의미
<code>orbit</code>	플레이어 주위를 선회
<code>chase</code>	표적을 추적
<code>dash</code>	표적으로 돌진
<code>zigzag</code>	갈지자로 접근
<code>hold</code>	제자리 대기
<code>retreat</code>	표적 반대로 후퇴

"A 하다가 B" 같은 순차 묘사는 `steps` 배열의 순서로 표현된다. 예를 들어 "**지그재그로 접근하다 돌진**" → `[zigzag, dash]`:

```
"behavior": {
  "steps": [
    { "kind": "zigzag", "seconds": 2, "speed": 300, "amplitude": 60 },
    { "kind": "dash", "seconds": 1, "speed": 440 }
  ],
  "loop": false
}
```

여기서도 **코드가 상한을 강제**한다: `seconds 1~6`, `speed 1~460`, `orbit.radius 1~150`, `zigzag.amplitude 1~100`, `steps` 최대 6개. LLM이 과한 수치를 뱉어도 밸런스가 깨지지 않는다. 이 DSL 덕분에 플레이어는 "정해진 소환수"가 아니라 **자기**가 **말로 설계한 움직임**을 보게 된다 — "LLM이 게임 행동을 설계한다"가 한 컷에 보이는 지점이다.

벽 형상 DSL (프롬프트 6항)

움직임(behavior)이 "어떻게 움직이는가"를 열었다면, 같은 원리로 벽(form: wall)의 모양도 플레이어의 말로 설계한다. "화염의 벽을 지그재그로 세워라"에서 "지그재그"는 예전엔 조용히 버려졌다(벽이 고정 원호 하나뿐이었다). 이제 LLM이 shape를 함께 설계한다:

- 형상 6종: arc (원호)·line (직선)·zigzag (갈지자)·wave (물결)·ring (단힌 원)·polygon (다각형).
- "원을 그리며 둘러싸라" → ring, "삼각형으로" → polygon. 특정 단어를 하드코딩하지 않고 **모양 자체를 어휘로** 연다.
- 같은 안전벽: 화이트리스트 밖은 거부→기본 원호 폴백, 수치 클램프. **형상은 위력이 아니다** — 어떤 모양이든 벽이 막는 정면폭은 크기(size)가 정한 값으로 정규화된다(모양은 표현일 뿐).

이 기능은 "만들기 전에 측정한다" 원칙으로 검증했다: 실 Gemini 배포 후 모양 묘사 문장의 **형상 반영 10/10**, 모양 묘사가 없을 때 오탐 **0/6**(원호 유지). "LLM이 게임의 형태를 설계한다"가 소환 행동에 이어 벽에서도 성립한다.

1.6 복합 영창 시퀀스

단일 SpellSpec 만으로는 "물러섰다가 화염 폭풍을 부른다"처럼 **시간 순서가 있는 문장**이나 "얼음과 번개를 동시에 내리꽂는다"처럼 **동시 행동이 있는 문장**을 보존하기 어렵다. 현재 판정기는 이런 입력에만 spell_plan을 내며, 토큰을 아끼기 위해 복합 판정의 대표 spell은 생략한다. 클라이언트가 plan에서 기록·각인용 대표 스펙을 결정론적으로 유도한다.

```
{
  "spell_plan": {
    "name": "후퇴의 화염진",
    "power": 0,
    "durationMs": 1500,
    "sequences": [
      { "behaviors": [
        { "type": "move", "destination": "away-from-target", "element": "fire" }
      ] },
      { "behaviors": [
        { "type": "form", "powerWeight": 1,
          "spec": { "name": "화염 폭풍", "effect": "damage", "target": "area",
                    "element_primary": "fire", "element_secondary": null,
                    "form": "nova", "size": "large", "speed": "normal",
                    "status": ["burn"], "power": 0, "cost": 0 } }
        ] }
    ]
  }
}
```

- sequences는 순차 실행하고, 한 단계 안의 behaviors는 병렬 실행한다. 행동 부품은 form·move·wait 세 종류뿐이다.
- Worker가 생성하는 이동 목적지는 공개 계약의 5종으로 제한한다. 로컬 런타임은 쇼케이스·향후 조준용 3종까지 포함해 8종을 검증하지만, 원격 모델의 자유도를 넓힌 것은 아니다.
- validateSpellPlan과 로컬 정규화가 최대 10단계·단계당 5행동·총 3초를 강제하고, 이동 하나당 전체 Power의 10%를 소비시킨다. 개별 피해·마나·픽셀 이동량은 LLM 값으로 실행하지 않는다.
- 배포 게이트에서 복합·단일 구조 정확도는 100%, held-out 핵심 행동 보존율은 88%, 2.5초 초과율은 0%(최대 1.99초)였다. 원격 판정이 실패해도 MockJudge가 명시적 순차 표현을 간이 plan으로 보존한다.
- 긴급 롤백은 VITE_SEQUENCE_JUDGE=0으로 plan 소비만 끄면 된다. 기존 v2 단일 주문 경로는 계속 유지한다.

1.7 기억하는 보스 — 도발 대사 생성

최종 보스는 플레이어의 **지난 전적을 기억**하고, LLM이 그에 맞는 도발 대사를 생성한다. 프록시의 /boss-line 엔드포인트가 런 요약(JSON)을 받아 1~2문장의 대사를 돌려준다.

```
플레이어 전적: { favoriteElement: "fire", topSpellName: "화염 대폭발",
                deaths: 3, clears: 0 }
|
▼ (/boss-line, LLM)
보스 대사: "또 불이나. 화염 대폭발도 세 번은 통하지 않아."
```

프롬프트는 애용 원소·최고 주문명·사망 횟수를 자연스럽게 비교도록, 첫 조우(사망·클리어 모두 0)면 낫선 도전자를 알보는 톤으로 지시한다. 실패 시 클라이언트가 템플릿 대사로 폴백하므로 여기서도 무중단이 보장된다.

1.8 진화·융합 작명

주문이 진화하거나 정령이 융합할 때, 격상된 이름을 LLM이 짓는다(/evolve-name). 예: fire+lightning 융합 → "작열하는 뇌운". 12자 이내의 함축적 이름 하나만 생성하며, 이 결과도 localStorage에 캐시된다(같은 조합 = 같은 이름, 재현성·호출 절감). 실패 시 템플릿 이름으로 폴백한다.

1.9 인프라 — Cloudflare Worker 프록시

모든 LLM 호출은 Cloudflare Worker 프록시(proxy/worker.js)를 경유한다. 프록시의 역할은 네 가지:

1. **API 키 은닉** — Gemini API 키는 워커 시크릿(GEMINI_API_KEY)에 있고 클라이언트에 절대 노출되지 않는다.
2. **레이트리밋** — IP·인스턴스별 분당 15회(RATE_LIMIT_PER_MIN = 15)로 남용을 막는다. 이는 앱 자체 보호막이며, Gemini 프로젝트의 실제 할당량은 모델·사용 티어별 AI Studio 활성 한도를 따른다.
3. **CORS** — 배포 오리진(GitHub Pages)과 로컬 개발(vite dev, 유동 포트) 요청만 허용한다.
4. **프롬프트 서버측 고정** — 1.3~1.8의 모든 프롬프트가 워커에 있다.

모델 핀 정책 (중요): 모델은 gemini-3.5-flash-lite 로 명시 고정한다. 초기에 gemini-flash-lite-latest 별칭을 썼다가, 이 별칭이 심사 기간 중 2.5 → 3.1 → 3.5 로 자동 드리프트하며 요청 규격(예: Gemini 3.5부터 temperature / thinkingBudget 폐기)이 바뀌는 문제를 겪었다. 재현성을 위해 -latest 별칭을 금지하고 버전을 못박았다. (이 사건의 디버깅 교훈은 2부에서 상술)

판정 요청의 생성 설정은 maxOutputTokens: 2048, responseType: 'application/json' 이다. 모델이 코드펜스(``json) 등 군더더기를 붙일 수 있으므로, 워커는 응답에서 첫 { ~마지막 } 구간만 추출해 파싱을 확인한 뒤 반환한다 (검증은 클라이언트에서 한 번 더).

2부. AI로 만든 게임

INCANT는 "AI를 게임에 넣은" 것에 그치지 않고, **게임 자체를 AI 에이전트로 만들었다**. 3인 팀이 각자 코딩 에이전트를 지휘해 판정 시스템·전투·연출·아트·사운드를 구현했고, 모든 유의미한 작업을 `docs/AI_USAGE_LOG.md`에 즉시 1줄씩 기록했다 — 이 글을 쓰는 시점 **76건**이 누적돼 있다.

2.1 사용한 AI 도구

도구	용도	주 담당 영역
Claude Code (Opus 4.8)	코드 구현·디버깅·아트 파이프라인	AI 판정 시스템, 시각 연출, 시스템 코드
OpenAI Codex	코드 구현	런 구조, 전투 폼, 보스 패턴, 통합 폴리싱
Google Gemini (Nano Banana 2)	이미지 생성	적·플레이어 스프라이트, 보스방 배경
Adobe Firefly	오디오 생성	효과음(Generate Sound Effects), BGM(Generate Soundtrack)

로그 76건의 도구 분포는 **Codex 계열 40건**, **Claude Code 계열 36건**으로, 두 코딩 에이전트를 병행했다. 이미지는 Gemini, 사운드는 Firefly가 전담했다(각 상세는 3부).

2.2 협업 워크플로

- 에이전트 병렬 지휘**: 3인이 각자 다른 에이전트(Claude Code / Codex)를 동시에 몰며, GitHub 이슈·PR로 작업을 분리했다.
- 계약 우선(contract-first)**: 시스템 간 인터페이스(예: 보상 종류 `RewardKind`, 판정 스키마)를 먼저 커밋해 계약을 고정하고, 그 위에서 각자 병렬로 구현했다 — 에이전트 간 충돌을 줄이는 핵심.
- 회귀로 방어**: 각 기능마다 순수 로직을 회귀 스크립트로 고정(`test:judge`, `test:grimoire`, `test:spirit` 등)해, 다른 에이전트의 변경이 기존 계약을 깨면 즉시 드러나게 했다.
- 즉시 로깅**: "나중에 몰아 쓰면 디테일이 사라진다"는 규칙 아래, 작업 직후 프롬프트 요약·산출물·교훈을 로그에 남겼다.

2.3 대표 사례 — 프롬프트에서 산출물까지

로그에서 네 가지 사례를 골랐다. 공통점은 **"AI가 낸 결과를 곧이곧대로 믿지 않고, 실제 값으로 검증했다"**는 것이다.

① **판정기 안정화 — 모델 드리프트와 인코딩 오진** 판정 프록시가 **"화염구"**에도 `fizzle`을 내는 위기를 겪었다. 원인 추적 중 두 가지가 겹쳐 있었다: (a) `gemini-flash-lite-latest` 별칭이 **2.5→3.1→3.5**로 자동 드리프트하며 폐기된 모델이 404를 냈고, (b) **Windows Git Bash**에서 `curl`로 한글을 인라인 전송하면 **UTF-8이 깨져** 서버가 깨진 문자열을 년센스로 판정하고 있었다. 후자를 "모델이 짧은 한글을 못 읽는다"로 오진해 여러 날 모델 교체를 헛돌았다. **교훈**: 게임(브라우저)은 항상 정상 UTF-8을 보내 실제로는 멀쩡했다. Node로 파일 바디를 만들어 재현하니 프록시가 9/9 정확히 판정 — **테스트 도구 자체가 오염원**이었다. 이후 모델을 `gemini-3.5-flash-lite`로 명시 편했다.

② **스프라이트 누끼 — "Gemini는 투명 PNG를 못 준다"** 몸·플레이어 스프라이트 7종을 Gemini로 생성했는데, **모든 출력이 알파 0%**(투명 대신 체커보드를 픽셀로 그려 옴)였다. Claude Code가 배경 톤을 자동 검출하고 테두리부터 flood-fill로 누끼를 따 해결했다. 또 총괄의 지적 **"틴트 때문에 몸이 그 색으로만 보인다"**가 정확해서 — `setTint`는 곱셈이라 재질 감이 죽는다 — **재질 텍스처와 발광 마스크를 2장으로 분리**해, 색은 발광이 전담하고 재질은 보존하도록 렌더 파이프라인을 바꿨다(`src/render/spriteLayers.ts`).

③ **프롬프트 튜닝의 편향 정정 — held-out 검증** 판정 프롬프트를 튜닝하던 중, 사용자가 **"네 프롬프트 예시에만 반응하게 편향된 것 아니냐"**고 지적했다. 재점검하니 프롬프트에 코퍼스 문구(**숲의 분노**)를 예시로 박아두고 **같은 문구로 검증**하는 teaching-to-the-test였다. 예시를 제거하고 순수 원리로 교체한 뒤, **프롬프트·코퍼스에 없는 held-out 3쌍**으로 재검

증하니 교차언어 위력 격차가 5.0(오염)→3.3(무오염)으로 드러났다. **교훈**: LLM 품질 주장은 튜닝 예시와 분리된 데이터로, 다회 평균으로 검증해야 한다.

④ **배경이 안 뜨던 근본 원인 — 로더 경로 오염** 배경이 안 뜨는 버그를 포맷 문제로 오판해 webp→jpg→png로 세 번 바뀌도 실패했다. network 탭에서 **Phaser가 실제로 요청한 URL**을 보고서야 원인을 잡았다: 오디오 로더가 설정한 `setPath`가 되돌려지지 않아 배경 요청 경로가 오염됐고, 그 경로에 Vite가 `index.html`을 200으로 반환해 Phaser가 HTML을 이미지로 처리하려다 실패한 것이었다. **교훈**: `fetch`가 200이라고 믿지 말고 로더가 실제로 요청한 URL을 볼 것.

2.4 배운 점 — 에이전트 지휘 노하우

- **"초록불"을 믿지 말고 실제 값으로 검증한다.** 통과하는 테스트가 아니라 held-out 데이터·다회 평균·network 탭 실측·실제 조회로 의도가 맞는지 본다. 위 4개 사례가 전부 "그럴듯한 결과"를 한 겹 더 파고들어 잡은 것이다.
 - **도구 경계에 함정이 있다.** Windows `curl`의 UTF-8, Gemini의 알파 부재, `fetch` 200의 착시 — 버그는 종종 우리 코드가 아니라 도구와 코드의 틈에 숨는다.
 - **창의성은 LLM에, 밸런스·안전·재현성은 코드에.** 게임 안에서든(1부) 개발 과정에서든 같은 원칙을 지켰다 — AI의 판단은 반드시 검증·클램프를 거친다.
 - **계약을 먼저 고정하면 에이전트를 병렬로 몰 수 있다.** 인터페이스를 커밋으로 못박고 회귀로 지키는 것이, 여러 에이전트의 동시 작업을 충돌 없이 합치는 열쇠였다.
-

3부. 외부 에셋·오픈소스 출처

전체 에셋별 생성 프롬프트·선정 근거·라이선스 원문·후처리 상세는 저장소의 `docs/ASSET_CREDITS.md`에 기록돼 있다. 아래는 출처·라이선스 요약이다.

3.1 오디오 — Adobe Firefly

- **효과음**: Adobe Firefly — *Generate Sound Effects*
- **BGM**: Adobe Firefly — *Generate Soundtrack (Beta)*
- **라이선스**: Adobe 공식 안내상 Generate Soundtrack 결과물은 royalty-free·상업적 사용 가능하며, Firefly FAQ는 Beta 기능도 별도 금지가 없는 한 상업 프로젝트에 사용 가능하다고 안내한다. (이용 조건 확인일 2026-07-17)
 - 효과음: <https://www.adobe.com/products/firefly/features/sound-effect-generator.html>
 - 사운드트랙: <https://www.adobe.com/products/firefly/features/ai-music-generator.html>
 - Firefly Beta 상업 이용: <https://helpx.adobe.com/firefly/web/get-started/learn-the-basics/adobe-firefly-faq.html>
- **후처리**: 저장소의 `scripts/process-audio-assets.py` · `scripts/create-bgm-loop.py`로 무음 정리·페이드·피크 정규화·루프 제작. 게임 경로 `public/assets/audio/`.

3.2 이미지·스프라이트 — Google Gemini (Nano Banana 2)

- **용도**: 적 6종·플레이어 스프라이트, 보스방/스테이지 배경. 게임 경로 `public/assets/sprites/`, `public/assets/backgrounds/`.
- **생성**: 게임 실제 엔티티와 탑다운 시점·색 구분 체계에 맞춘 프롬프트를 Claude Code가 설계하고 Gemini로 생성.
- **후처리(Claude Code)**: ① 출력 우하단의 Gemini 워터마크를 거울 패치 페더링으로 제거 ② 투명 대신 그려진 체크보드 배경을 누끼 처리 ③ 재질/발광 텍스처 2장 분리. AI 생성 원본은 별도 보관하고 저장소에는 채택본만 포함.

3.3 폰트 — Noto Serif KR

- **용도**: 타이틀 카피·주문명 각인.
- **라이선스**: **SIL Open Font License 1.1**. 원문을 `public/assets/fonts/OFL-NotoSerifKR.txt`에 포함.
- **출처**: <https://github.com/google/fonts/tree/main/ofl/notoserifkr> (Google Fonts 공식 가변 TTF에서 필요한 자모만 서브셋한 WOFF2를 자체 호스팅).

3.4 프로젝트 자체 제작

- `public/assets/og-incant.svg`: 타이틀 화면의 색상·마법진·타이포그래피를 재사용한 1200×630 OG 이미지 — 팀 자체 제작.

3.5 오픈소스 라이브러리

라이브러리	버전	라이선스	용도
Phaser	3.90	MIT	게임 엔진(렌더·입력·씬)
TypeScript	5.9	Apache-2.0	언어·타입 검사
Vite	7.1	MIT	빌드·개발 서버

인프라: **Cloudflare Workers**(판정 프록시 호스팅), **GitHub Pages**(웹 빌드 배포).